

GRAND CHALLENGE MONOGRAPH COLLECTION

COMPREHENSION

Complete Solutions and Rubrics Companion

Volume II

Grand Challenge Labs

How to use this companion

Every one of the 168 teaching exercises has a keyed assessment entry. Short factual and construction exercises are accompanied by an expected method/result criterion; open-ended Design Clinic and Challenge exercises receive explicit rubrics rather than fake unique solutions. This companion is pedagogical material, not external mathematical review.

Contents

How to use this companion	ii
1 Chapter 1 solutions and rubrics	1
2 Chapter 2 solutions and rubrics	5
3 Chapter 3 solutions and rubrics	9
4 Chapter 4 solutions and rubrics	13
5 Chapter 5 solutions and rubrics	17
6 Chapter 6 solutions and rubrics	21
7 Chapter 7 solutions and rubrics	25
8 Chapter 8 solutions and rubrics	29
9 Chapter 9 solutions and rubrics	33
10 Chapter 10 solutions and rubrics	37
11 Chapter 11 solutions and rubrics	40
12 Chapter 12 solutions and rubrics	44
13 Chapter 13 solutions and rubrics	48
14 Chapter 14 solutions and rubrics	51

Chapter 1

Chapter 1 solutions and rubrics

1.1 — Checkpoint

Exercise. Decide which is well formed: $n : \mathbb{N}, i : \text{Fin}(n)$ or $i : \text{Fin}(n), n : \mathbb{N}$. Name the failed premise in the bad case.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

1.2 — Checkpoint

Exercise. Derive $n : \mathbb{N} \vdash \text{Fin}(\text{succ}(n))$ type.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

1.3 — Checkpoint

Exercise. Explain why family formation is not a runtime case split.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

1.4 — Core

Exercise. Give a four-declaration telescope containing two declarations that depend on the same natural-number variable.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

1.5 — Core

Exercise. Show by weakening that $n : \mathbb{N}, b : \text{Bool} \vdash \text{Fin}(n)$ type.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

1.6 — Core

Exercise. Construct an attempted exchange of adjacent declarations that breaks well-formedness.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

1.7 — Synthesis

Exercise. Compare a telescope with a compiler symbol table. Which implementation details may differ while the same typing invariant is preserved?

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

For Π , remember that application specializes the codomain by substitution and that a constant codomain recovers the ordinary arrow in this presentation.

1.8 — Synthesis

Exercise. Refine the slogan “types can contain values” into a statement that is defensible in COMP-0.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

1.9 — Proof Workshop

Exercise. Prove the variable case of weakening, distinguishing a variable declared in Γ from one declared in Δ .

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

1.10 — Proof Workshop

Exercise. Prove that if $\Gamma \vdash t : \mathbb{N}$, then extending Γ by a fresh independent Boolean declaration preserves formation of $\text{Fin}(t)$.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

1.11 — Design Clinic

Exercise. Add $\text{Matrix}(m, n)$ only as a family former. State its formation rule and three contexts it makes expressible.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

1.12 — Challenge

Exercise. State a sufficient free-variable condition under which two adjacent declarations in a telescope may be exchanged.

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

Chapter 2

Chapter 2 solutions and rubrics

2.1 — Checkpoint

Exercise. Compute $(\text{Fin}(\text{succ}(n)))[0/n]$.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

2.2 — Checkpoint

Exercise. Substitute 2 for n in $n : \mathbb{N}, i : \text{Fin}(n), j : \text{Fin}(\text{succ}(n))$.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

2.3 — Checkpoint

Exercise. State why substitution and evaluation are distinct.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

2.4 — Core

Exercise. Give an example where substitution changes a trailing declaration type but not the term being judged.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

2.5 — Core

Exercise. Check the substitution-composition equation for $E = \text{Fin}(\text{succ}(x))$.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

2.6 — Core

Exercise. Explain why a theorem stated only for $\Gamma, x : A \vdash J$ is inconvenient beneath later dependent binders.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

2.7 — Synthesis

Exercise. Interpret generalized substitution as specialization of a family of contexts.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

2.8 — Synthesis

Exercise. Explain which part of substitution is syntactic and which part becomes semantic only after a model is chosen.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

2.9 — Proof Workshop

Exercise. Prove the Fin -formation case of generalized substitution.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

2.10 — Proof Workshop

Exercise. Prove substitution composition for variables and successor terms.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not

assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

2.11 — Design Clinic

Exercise. Extend the substitution function to the family $\text{Matrix}(m, n)$ and list the new test cases required.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

2.12 — Challenge

Exercise. State the capture-avoidance problem that will appear once binders are added, and give the alpha-renaming remedy.

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

Chapter 3

Chapter 3 solutions and rubrics

3.1 — Checkpoint

Exercise. State the Π -formation rule.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

For Π , remember that application specializes the codomain by substitution and that a constant codomain recovers the ordinary arrow in this presentation.

3.2 — Checkpoint

Exercise. When may $\Pi(x : A).B$ be written as $A \rightarrow B$ in this volume?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

For Π , remember that application specializes the codomain by substitution and that a constant codomain recovers the ordinary arrow in this presentation.

3.3 — Checkpoint

Exercise. If $f : \Pi(n : \mathbb{N}).\text{Fin}(\text{succ}(n))$, derive the type of $f0$.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

For Π , remember that application specializes the codomain by substitution and that a constant codomain recovers the ordinary arrow in this presentation.

3.4 — Core

Exercise. Type $\lambda n.\text{succ}(n)$ as a dependent function whose codomain happens to be constant.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

3.5 — Core

Exercise. Give a Π -type that cannot be expressed faithfully as one fixed arrow type without hiding the dependency.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

For Π , remember that application specializes the codomain by substitution and that a constant codomain recovers the ordinary arrow in this presentation.

3.6 — Core

Exercise. Perform capture-avoiding substitution in $\lambda y.x$ with y substituted for x .

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

3.7 — Synthesis

Exercise. Explain why dependent application is simultaneously elimination and specialization.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

3.8 — Synthesis

Exercise. Contrast “one function, many codomains” with overloading in an ordinary programming language.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

3.9 — Proof Workshop

Exercise. Give the one-line proof of principal beta subject reduction from generalized substitution.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

3.10 — Proof Workshop

Exercise. Show that the ordinary simply typed beta rule is recovered when the codomain is constant.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

3.11 — Design Clinic

Exercise. Specify a dependent function type for matrix transpose that records input and output dimensions, without defining matrices themselves.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

3.12 — Challenge

Exercise. Explain why adding function extensionality to definitional equality would materially change the checking problem, even though it sounds mathematically natural.

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

Chapter 4

Chapter 4 solutions and rubrics

4.1 — Checkpoint

Exercise. State the Σ -formation rule.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

For Σ , the second component/projection is checked in the family member selected by the first component.

4.2 — Checkpoint

Exercise. State the typing rule for the second projection.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

For Π , remember that application specializes the codomain by substitution and that a constant codomain recovers the ordinary arrow in this presentation.

4.3 — Checkpoint

Exercise. When is $\Sigma(x : A).B$ an ordinary product in this presentation?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

For Σ , the second component/projection is checked in the family member selected by the first component.

4.4 — Core

Exercise. Type a pair $\langle 2, i \rangle$ assuming $i : \text{Fin}(2)$.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

4.5 — Core

Exercise. Explain why $\text{snd } p$ cannot in general be assigned one fixed type independent of $\text{fst } p$.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

4.6 — Core

Exercise. Write the two projection beta equations.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

4.7 — Synthesis

Exercise. Compare a dependent pair with a dynamically tagged union. What information is similar, and what is formally different?

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

4.8 — Synthesis

Exercise. Explain the computational content of the future existential reading of Σ .

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

For Σ , the second component/projection is checked in the family member selected by the first component.

4.9 — Proof Workshop

Exercise. Prove preservation of the first projection beta step.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

4.10 — Proof Workshop

Exercise. Prove preservation of the second projection beta step, naming the conversion used.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

4.11 — Design Clinic

Exercise. Design a Σ type for a matrix packaged with dimensions that its payload must respect.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

For Σ , the second component/projection is checked in the family member selected by the first component.

4.12 — Challenge

Exercise. Why does a dependent record system need field-order or dependency information even when users see named fields?

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

Chapter 5

Chapter 5 solutions and rubrics

5.1 — Checkpoint

Exercise. What premise licenses the extension $\Gamma, x : A$?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

5.2 — Checkpoint

Exercise. What does the projection $\Gamma.A \rightarrow \Gamma$ forget?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

5.3 — Checkpoint

Exercise. State one condition that blocks exchange.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

5.4 — Core

Exercise. Determine all legal adjacent swaps in $n : \mathbb{N}, b : \text{Bool}, i : \text{Fin}(n)$.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible. Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

5.5 — Core

Exercise. Reindex $n : \mathbb{N}, i : \text{Fin}(n), j : \text{Fin}(\text{succ}(n))$ along $2/n$.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible. Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

5.6 — Core

Exercise. Identify the common family data consumed by both Π and Σ formation.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

For Π , remember that application specializes the codomain by substitution and that a constant codomain recovers the ordinary arrow in this presentation.

For Σ , the second component/projection is checked in the family member selected by the first component.

5.7 — Synthesis

Exercise. Explain why comprehension is a better organizing idea than a list of unrelated dependent type constructors.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

5.8 — Synthesis

Exercise. Give a programming-language analogy for context projection and state where it breaks down.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

5.9 — Proof Workshop

Exercise. Justify adjacent exchange under the stated independence conditions.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

5.10 — Proof Workshop

Exercise. Explain how weakening and substitution together maintain well-formed trailing contexts.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

5.11 — Design Clinic

Exercise. Propose a new family former and list the structural obligations it creates before any evaluator is written.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

5.12 — Challenge

Exercise. Formulate context morphisms as simultaneous substitutions and explain why composition should correspond to substitution composition.

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

Chapter 6

Chapter 6 solutions and rubrics

6.1 — Checkpoint

Exercise. Why is there no constructor for $\text{Fin}(0)$?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

6.2 — Checkpoint

Exercise. Give the type of $\text{fsucc}(\text{fzero})$ at its smallest possible bound.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

6.3 — Checkpoint

Exercise. What fact is recorded by the index of $\text{Fin}(n)$?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

6.4 — Core

Exercise. Enumerate the canonical inhabitants of $\text{Fin}(3)$ using the two constructors.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

6.5 — Core

Exercise. Show that a closed value of $\text{Fin}(1)$ must be fzero .

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

6.6 — Core

Exercise. Give an invariant that $\text{Fin}(n)$ does not encode.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

Keep code and decoded type distinct and respect the stated stratification boundary; no same-level self-code is admitted.

6.7 — Synthesis

Exercise. Explain why constructor result types, not prose documentation, make the bound discipline part of the language.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

6.8 — Synthesis

Exercise. Compare an indexed state with a runtime assertion. When might an implementation still retain a runtime check?

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

6.9 — Proof Workshop

Exercise. Prove the canonical-forms lemma for $\text{Fin}(\text{succ}(n))$ by inversion on values.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

6.10 — Proof Workshop

Exercise. Prove there is no closed canonical value of $\text{Fin}(0)$ in the selected constructor grammar.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

6.11 — Design Clinic

Exercise. Design a three-phase protocol family indexed by a finite phase label. State only constructor result types.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

6.12 — Challenge

Exercise. Explain why canonical forms plus preservation is still insufficient to prove termination.

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

Chapter 7

Chapter 7 solutions and rubrics

7.1 — Checkpoint

Exercise. Give the type of `vnil`.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

7.2 — Checkpoint

Exercise. Give the type of `vcons(a, xs)` from the premises $a : A$ and $xs : \text{Vec}(A, n)$.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Track the vector length index through `nil/cons` and use constructor inversion when a shape is excluded by the index.

7.3 — Checkpoint

Exercise. Why can a value of `Vec(A, 0)` not be a `vcons` under the current equality theory?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Track the vector length index through `nil/cons` and use constructor inversion when a shape is excluded by the index.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

7.4 — Core

Exercise. Type a three-element vector explicitly from the constructors.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the vector length index through `nil/cons` and use constructor inversion when a shape is excluded by the index.

7.5 — Core

Exercise. Show the failed inversion when `vnil` is claimed to inhabit $\text{Vec}(A, 2)$.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the vector length index through `nil/cons` and use constructor inversion when a shape is excluded by the index.

7.6 — Core

Exercise. Compare the constructor types of a list and a vector.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the vector length index through `nil/cons` and use constructor inversion when a shape is excluded by the index.

7.7 — Synthesis

Exercise. Explain how constructor result indices encode an invariant without a separate length theorem.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

Keep code and decoded type distinct and respect the stated stratification boundary; no same-level self-code is admitted.

7.8 — Synthesis

Exercise. Why is the selected-family discipline scientifically cleaner than claiming a general positivity checker?

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

7.9 — Proof Workshop

Exercise. Prove the zero-length branch of the vector canonical-forms lemma.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Track the vector length index through nil/cons and use constructor inversion when a shape is excluded by the index.

7.10 — Proof Workshop

Exercise. Prove the successor-length branch by constructor inversion.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

7.11 — Design Clinic

Exercise. Design an indexed binary tree family whose index records height exactly; state constructor result types and note one design difficulty.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

7.12 — Challenge

Exercise. Explain how definitional equality of indices can affect constructor inversion.

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

Chapter 8

Chapter 8 solutions and rubrics

8.1 — Checkpoint

Exercise. State a type for vector head that excludes the empty case.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Track the vector length index through nil/cons and use constructor inversion when a shape is excluded by the index.

8.2 — Checkpoint

Exercise. State the type of safe lookup.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

8.3 — Checkpoint

Exercise. What is a motive?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

8.4 — Core

Exercise. Reduce lookup on a two-element vector at `fzero`.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the vector length index through `nil/cons` and use constructor inversion when a shape is excluded by the index.

8.5 — Core

Exercise. Reduce lookup on the same vector at `fsucc(fzero)`.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the vector length index through `nil/cons` and use constructor inversion when a shape is excluded by the index.

8.6 — Core

Exercise. Explain why lookup on an empty vector has no canonical well-typed index argument.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the vector length index through `nil/cons` and use constructor inversion when a shape is excluded by the index.

8.7 — Synthesis

Exercise. Compare dependent elimination with ordinary case analysis.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

8.8 — Synthesis

Exercise. Explain exactly what totality guarantee the vector-head type provides and what it does not.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

Track the vector length index through nil/cons and use constructor inversion when a shape is excluded by the index.

8.9 — Proof Workshop

Exercise. Derive the no-empty-vector-head proposition from vector canonical forms.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Track the vector length index through nil/cons and use constructor inversion when a shape is excluded by the index.

8.10 — Proof Workshop

Exercise. Show that the recursive lookup branch decreases both vector length and finite-index structure in lockstep.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

Track the vector length index through nil/cons and use constructor inversion when a shape is excluded by the index.

8.11 — Design Clinic

Exercise. Specify a safe matrix row-selection function using a finite row index.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

8.12 — Challenge

Exercise. Give an example where an indexed family requires a genuinely dependent motive rather than one fixed result type.

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

Chapter 9

Chapter 9 solutions and rubrics

9.1 — Checkpoint

Exercise. State the conversion rule.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

9.2 — Checkpoint

Exercise. Name two principles not included in the volume’s definitional equality.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

9.3 — Checkpoint

Exercise. Why can index computation affect type checking?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

9.4 — Core

Exercise. Normalize $(\lambda x.\text{succ}(x))0$ and explain the induced vector-type conversion.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the vector length index through `nil/cons` and use constructor inversion when a shape is excluded by the index.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

9.5 — Core

Exercise. Give a mathematical equality that need not be definitional in a chosen recursive presentation.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the finite bound in every constructor or family-formation step; `Fin(0)` has no admitted canonical constructor.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

9.6 — Core

Exercise. Explain the difference between alpha-equivalence and propositional equality.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

9.7 — Synthesis

Exercise. Why is definitional equality a language-design budget rather than merely a mathematical relation?

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

9.8 — Synthesis

Exercise. Describe three possible responses when a useful mathematical equality is not definitional.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

9.9 — Proof Workshop

Exercise. Show that the principal Π beta equality is stable under formation of $\text{Vec}(A, -)$ by congruence.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Track the vector length index through nil/cons and use constructor inversion when a shape is excluded by the index.

For Π , remember that application specializes the codomain by substitution and that a constant codomain recovers the ordinary arrow in this presentation.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

9.10 — Proof Workshop

Exercise. Explain why conversion soundness depends on the equality relation preserving type formation.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

9.11 — Design Clinic

Exercise. Choose an equality policy for a small tensor-shape language and justify what you normalize automatically.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

9.12 — Challenge

Exercise. Analyze the trade-off of adding eta equality for functions to the kernel's algorithmic equality.

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

Chapter 10

Chapter 10 solutions and rubrics

10.1 — Checkpoint

Exercise. Which judgment synthesizes a type and which checks against an expected type?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

10.2 — Checkpoint

Exercise. Why is application naturally a synthesis rule?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

10.3 — Checkpoint

Exercise. Why does an unannotated lambda usually check rather than synthesize?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

10.4 — Core

Exercise. Derive the algorithmic type of fa from a synthesized dependent function type.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

10.5 — Core

Exercise. Show how an annotation lets a lambda participate in a synthesis context.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

10.6 — Core

Exercise. Give a dependent pair that must be checked against an expected Σ type.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

For Σ , the second component/projection is checked in the family member selected by the first component.

10.7 — Synthesis

Exercise. Explain bidirectionality as information routing rather than a new notion of truth.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

10.8 — Synthesis

Exercise. Identify one declaratively typable term that the deliberately incomplete checker might reject without extra annotation.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

10.9 — Proof Workshop

Exercise. Prove algorithmic soundness for the application rule.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

10.10 — Proof Workshop

Exercise. Prove algorithmic soundness for lambda checking.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

10.11 — Design Clinic

Exercise. Decide which constructs in a small dependent record language should synthesize and which should check.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

10.12 — Challenge

Exercise. Explain why completeness of elaboration is a materially stronger claim than soundness of the core checker.

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

Separate candidate reconstruction from kernel validity and distinguish ambiguity from search incompleteness and core invalidity.

Chapter 11

Chapter 11 solutions and rubrics

11.1 — Checkpoint

Exercise. What object does an elaborator produce for the kernel?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Separate candidate reconstruction from kernel validity and distinguish ambiguity from search incompleteness and core invalidity.

11.2 — Checkpoint

Exercise. What is a metavariable?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Separate candidate reconstruction from kernel validity and distinguish ambiguity from search incompleteness and core invalidity.

11.3 — Checkpoint

Exercise. Why is an occurs check needed?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

11.4 — Core

Exercise. Solve the constraints $\alpha = \mathbb{N}$ and $\text{Vec}(\alpha, 2) = \text{Vec}(\mathbb{N}, 2)$.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the vector length index through `nil/cons` and use constructor inversion when a shape is excluded by the index.

11.5 — Core

Exercise. Explain why $\alpha = \Pi(x : \mathbb{N}).\alpha$ must be rejected by an occurs check.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

For Π , remember that application specializes the codomain by substitution and that a constant codomain recovers the ordinary arrow in this presentation.

11.6 — Core

Exercise. Give an ambiguous omitted-argument example.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

11.7 — Synthesis

Exercise. Distinguish elaborator completeness from kernel soundness.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

Separate candidate reconstruction from kernel validity and distinguish ambiguity from search incompleteness and core invalidity.

11.8 — Synthesis

Exercise. Explain why a sophisticated untrusted search procedure can still be compatible with a small trusted checker.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

11.9 — Proof Workshop

Exercise. Prove the kernel trust-boundary proposition from the definition of acceptance.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

11.10 — Proof Workshop

Exercise. Show that first-order constructor decomposition preserves the solution set of an equality constraint.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

State whether the equality is definitional, algorithmic, alpha-equivalence, or a later propositional notion; do not silently move between them.

11.11 — Design Clinic

Exercise. Design an error report that distinguishes ambiguity, search incompleteness, and core invalidity.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

11.12 — Challenge

Exercise. Explain why dependent implicit-argument inference naturally approaches higher-order unification.

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

Chapter 12

Chapter 12 solutions and rubrics

12.1 — Checkpoint

Exercise. What are the two ingredients of a Tarski-style universe preview?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Keep code and decoded type distinct and respect the stated stratification boundary; no same-level self-code is admitted.

12.2 — Checkpoint

Exercise. State the decoding equations for natural and Boolean codes.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Keep code and decoded type distinct and respect the stated stratification boundary; no same-level self-code is admitted.

12.3 — Checkpoint

Exercise. Why is “ $\text{Type} : \text{Type}$ ” not adopted silently?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

12.4 — Core

Exercise. Design a code for ordinary products and state its decoding equation.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Keep code and decoded type distinct and respect the stated stratification boundary; no same-level self-code is admitted.

12.5 — Core

Exercise. Design a code for vectors when both the element type and length are explicit external parameters.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the vector length index through `nil/cons` and use constructor inversion when a shape is excluded by the index.

Keep code and decoded type distinct and respect the stated stratification boundary; no same-level self-code is admitted.

12.6 — Core

Exercise. Explain the difference between a code and the type it decodes to.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Keep code and decoded type distinct and respect the stated stratification boundary; no same-level self-code is admitted.

12.7 — Synthesis

Exercise. Explain how universes turn formation principles into manipulable data.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

Keep code and decoded type distinct and respect the stated stratification boundary; no same-level self-code is admitted.

12.8 — Synthesis

Exercise. Compare Russell-style and Tarski-style presentations at the level of intuition, without claiming them definitionally identical.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

12.9 — Proof Workshop

Exercise. Show by structural induction that the toy decoder is total on the admitted finite code grammar.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

Keep code and decoded type distinct and respect the stated stratification boundary; no same-level self-code is admitted.

12.10 — Proof Workshop

Exercise. Explain why rejecting self-level classification is a design condition rather than a full consistency proof.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

12.11 — Design Clinic

Exercise. Specify a two-level code hierarchy sufficient to code base types at level 0 and the level-0 code language at level 1.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

Keep code and decoded type distinct and respect the stated stratification boundary; no same-level self-code is admitted.

12.12 — Challenge

Exercise. Describe what additional metatheory would be needed before claiming a full internal universe hierarchy.

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

Keep code and decoded type distinct and respect the stated stratification boundary; no same-level self-code is admitted.

Chapter 13

Chapter 13 solutions and rubrics

13.1 — Checkpoint

Exercise. What does a context projection forget?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

13.2 — Checkpoint

Exercise. What operation corresponds syntactically to semantic reindexing?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

13.3 — Checkpoint

Exercise. Name one reason richer dependency can make checking harder.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

13.4 — Core

Exercise. Give an interface expressible directly with Π that cannot be stated as one fixed arrow without externalizing an invariant.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible. For Π , remember that application specializes the codomain by substitution and that a constant codomain recovers the ordinary arrow in this presentation.

13.5 — Core

Exercise. Identify the comprehension object associated with $\Gamma \vdash A$ type.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

13.6 — Core

Exercise. State the identity and composition laws expected of substitution/reindexing.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

13.7 — Synthesis

Exercise. Explain what is genuinely shared between telescope syntax and the semantics of a comprehension category.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

13.8 — Synthesis

Exercise. Defend or reject the statement “more expressive types are always better.”

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

13.9 — Proof Workshop

Exercise. Show how substitution identity/composition motivate functorial reindexing equations.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

Substitution acts through terms, types, and trailing telescope declarations; capture avoidance is mandatory under binders.

13.10 — Proof Workshop

Exercise. Explain why one categorical model does not by itself establish semantic completeness of the syntax.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

13.11 — Design Clinic

Exercise. Compare two type-language designs for tensor shapes: one with runtime checks, one with dependent indices. List benefits and costs.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

13.12 — Challenge

Exercise. State the strongest version of “formation rules build computational worlds” that the evidence of this volume supports without overclaiming.

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

Chapter 14

Chapter 14 solutions and rubrics

14.1 — Checkpoint

Exercise. Name four consequences of allowing type families.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

14.2 — Checkpoint

Exercise. State one guarantee an index can encode and one it cannot.

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

Keep code and decoded type distinct and respect the stated stratification boundary; no same-level self-code is admitted.

14.3 — Checkpoint

Exercise. What is the governing threshold into Volume III?

Key / rubric. A complete answer states the requested judgment/fact directly, names the formation, typing, conversion, or constructor rule that licenses it, and does not import stronger machinery from later chapters.

14.4 — Core

Exercise. Explain the future existential reading of $\Sigma(n : \mathbb{N}).\text{Vec}(A, n)$ without treating it as already formalized logic.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the vector length index through nil/cons and use constructor inversion when a shape is excluded by the index.

For Σ , the second component/projection is checked in the family member selected by the first component.

14.5 — Core

Exercise. Explain the future universal reading of $\Pi(x : A).B(x)$.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

For Π , remember that application specializes the codomain by substitution and that a constant codomain recovers the ordinary arrow in this presentation.

14.6 — Core

Exercise. List the trusted assumptions behind a claim that a finite index prevents out-of-bounds access.

Key / rubric. A complete answer exhibits the requested term, context, derivation, reduction, or counterexample explicitly and checks each dependency/index side condition. Equivalent constructions receive full credit when the typing obligations are made visible.

Track the finite bound in every constructor or family-formation step; $\text{Fin}(0)$ has no admitted canonical constructor.

14.7 — Synthesis

Exercise. State the strongest unification thesis supported by Volume II and one stronger claim it does not support.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

14.8 — Synthesis

Exercise. Explain why “every type is a proposition” is too crude for the series.

Key / rubric. A complete answer identifies the shared structure and the non-equivalence boundary. It must state at least one respect in which the comparison is exact and one respect in which it is only an analogy, implementation choice, or later semantic interpretation.

14.9 — Proof Workshop

Exercise. Match the Π introduction/elimination rules to the natural-deduction shapes expected for universal/implicational reasoning.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

For Π , remember that application specializes the codomain by substitution and that a constant codomain recovers the ordinary arrow in this presentation.

14.10 — Proof Workshop

Exercise. Match the Σ introduction/projection structure to a witness-and-evidence reading, naming the missing logical details.

Key / rubric. A complete proof names its induction or inversion measure, states the exact scoped premises used, treats the dependency-sensitive critical case, and does not assume normalization, extensionality, identity types, or completeness unless the chapter has explicitly admitted them.

For Σ , the second component/projection is checked in the family member selected by the first component.

14.11 — Design Clinic

Exercise. Choose one invariant from a real software system and describe how far an indexed type could enforce it before environmental assumptions enter.

Key / rubric. Rubric: give explicit formation/typing interfaces, state the invariant encoded, list the structural obligations created (substitution, weakening, conversion, elimination/checking), and identify at least one property deliberately left outside the type. Multiple designs may satisfy the rubric.

14.12 — Challenge

Exercise. Give a concrete counterexample to the naive thesis “if a property can be expressed as a dependent type, the resulting program is correct in every operational sense.”

Key / rubric. Rubric: the response must isolate the hard boundary raised by the question, give a technically defensible argument or counterexample, and distinguish theorem, implementation evidence, analogy, and open design choice. A single slogan is insufficient.

Completeness record

This companion contains 168 keyed entries: 12 for each of 14 teaching chapters. The machine-readable exercise audit records the same counts.